24ME311 PRODUCT DEVELOPMENT LAB-1

NALAMAI: AI-POWERED SMART HEALTHCARE ASSISTANT

A PROJECT REPORT

Submitted by

SAKTHI PRIYA S	111725201124
SHANMUGA PRIYA L	111725201143
SARANYAA D	111725201136

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M. Nagar, Kavaraipettai - 601 206



November 2026

R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M. Nagar, Kavaraipettai - 601 206

BONAFIDE CERTIFICATE

Certified that this project report "**NALAMAI: AI-POWERED SMART HEALTHCARE ASSISTANT"** is the bonafide work **of SAKTHI PRIYA S** (111725201124), **SARANYAA D** (111725201136) and **SHANMUGA PRIYA L** (111725201143) who carried out the project under my supervision.

SIGNATURE

Dr. Sandra Johnson M.E., Ph.D.,
Professor and Head
Department of Artificial Intelligence and Data Science,
R.M.K. Engineering College,
R.S.M. Nagar, Kavaraipettai,
Tiruvallur District – 601206.

SIGNATURE

Ms. Thenmozhi M.E.,(Ph.D.)
Assistant Professor and Faculty Incharge
Department of Artificial Intelligence and Data Science,
R.M.K. Engineering College,
R.S.M. Nagar, Kavaraipettai,
Tiruvallur District – 601206.

Submitted for the Product Development Lab-3 Examination held on ..... at R.M.K. Engineering College, Kavaraipettai, Tiruvallur District – 601206.

INTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to the management of R.M.K. Engineering College for providing the facilities and academic environment required to carry out this project.

We extend our thanks to the Principal and the Head of the Department of Artificial Intelligence and Data Science for their encouragement and support throughout the project work.

We would like to thank our faculty members and project guide for their guidance, suggestions, and valuable feedback during the preparation of the NalamAI project. Their support helped us organize the project concept and present its modules clearly.

Finally, we thank our parents and friends for their encouragement and support during the completion of this project report.

ABSTRACT

NalamAI is an AI-powered smart healthcare assistant proposed to improve access to healthcare services for people in rural and underserved communities. These communities may face challenges such as limited access to doctors, difficulty identifying health problems at an early stage, medication management, emergency response, and maintaining organized medical records. NalamAI brings several healthcare support functions together in one integrated platform.

The proposed system includes an AI-based voice symptom checker, multilingual voice and text support in Tamil and English, telemedicine support, medicine reminders, emergency SOS, caregiver notifications, and digital health records. Users can describe symptoms and receive preliminary health guidance before consulting a healthcare professional. The telemedicine module is intended to support remote access to doctors, while the reminder module helps users track medicine names, dosage, and timings.

In an emergency, the SOS feature can alert registered contacts and share the user's location. Digital records can maintain symptoms, previous consultations, medicines, and other health information for future reference. By combining artificial intelligence, voice technology, telemedicine, and mobile healthcare concepts, NalamAI aims to make healthcare more accessible, affordable, and convenient. The system is a support tool and does not replace professional diagnosis or emergency medical services. Future enhancements may include wearable health monitoring, offline functionality, and more personalized AI assistance.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	i
	ACKNOWLEDGEMENT	ii
	LIST OF FIGURES	iii
	LIST OF ABBREVIATIONS	iv
1	INTRODUCTION	1
	1.1 Problem Statement	1
	1.2 Objectives	2
	1.3 Scope of the Project	2
2	SYSTEM ANALYSIS	3
	2.1 Existing System	3
	2.2 Proposed System	4
	2.3 System Requirements Specification	5
3	LITERATURE SURVEY	6
	3.1 Review of Related Work	6
4	SYSTEM ARCHITECTURE AND DESIGN	10
	4.1 Architecture Overview	10
	4.2 Module Description	11
	4.3 Workflow	13
	4.4 Data Privacy and Safety	14
5	IMPLEMENTATION APPROACH	15
	5.1 Functional Requirements	15
	5.2 Testing and Validation	17
	5.3 Performance Metrics	18
6	CONCLUSION AND FUTURE ENHANCEMENTS	19
	REFERENCES	20
	APPENDIX I – ILLUSTRATIVE LOGIC	21

LIST OF FIGURES

S. No.	TITLE	PAGE NO.
1	NalamAI system architecture	10
2	NalamAI user workflow	13
3	Module interaction overview	12
4	Illustrative symptom-checker flow	21

LIST OF ABBREVIATIONS

S. No.	ABBREVIATION	EXPANSION
1	AI	Artificial Intelligence
2	SOS	Save Our Souls / Emergency alert
3	EHR	Electronic Health Record
4	UI	User Interface
5	UX	User Experience
6	NLP	Natural Language Processing
7	ICT	Information and Communication Technology
8	WHO	World Health Organization
9	SMS	Short Message Service
10	GPS	Global Positioning System

CHAPTER 1

INTRODUCTION

Healthcare access remains a challenge for people living in rural and underserved communities. Distance from hospitals, limited availability of doctors, language barriers, and difficulty maintaining medical information can make it harder for people to obtain timely support. Digital healthcare services can help reduce some of these barriers when they are designed to be accessible, understandable, and safe.

NalamAI is proposed as an integrated smart healthcare assistant that combines several useful services in one platform. Instead of requiring users to depend on separate applications for symptom guidance, telemedicine, medication reminders, emergency alerts, and medical records, the system brings these functions together. Voice and text interaction in Tamil and English is intended to make the platform more accessible to users who are more comfortable with regional-language communication.

1.1 PROBLEM STATEMENT

Existing healthcare technology solutions often focus on one specific function, such as online consultation, medicine reminders, symptom checking, or health-record storage. When these services are separate, users may need to switch between applications and repeat information. Rural users may also face language and connectivity barriers. These issues can delay access to information and make routine healthcare management more difficult.

The project therefore proposes NalamAI as a unified healthcare support platform that provides preliminary symptom guidance, remote doctor access, medicine reminders, emergency contact alerts, caregiver notifications, and digital medical records. The system is intended to support users and healthcare professionals, not to replace clinical judgement.

1.2 OBJECTIVES

- To propose an integrated healthcare assistant for rural and underserved communities.
- To provide preliminary symptom guidance through voice or text input.
- To reduce language barriers through Tamil and English support.
- To support remote access to healthcare professionals through telemedicine.
- To help users track medicine names, dosage, and timings.
- To support emergency alerts and caregiver notifications.
- To maintain organized digital health information for future reference.

1.3 SCOPE OF THE PROJECT

The scope of the Nalam AI project is to develop a user-friendly healthcare support web application that helps users organize their basic healthcare information and access preliminary health guidance in one place. The application provides several modules to improve accessibility and convenience.

1. Symptom Checker

The system allows users to enter their symptoms and receive preliminary, predefined healthcare guidance. In future versions, a validated AI-based model could be integrated to provide more personalized assistance.

2. Medicine Management

The application allows users to maintain medicine details, including medicine names, dosages, timings, and frequency. Future enhancements may include scheduled notifications and medicine adherence tracking.

3. Medical Record Management

Users can store and retrieve basic medical record details digitally. This feature can help organize healthcare information and make it easier to refer to previous records.

4. Caregiver Support

The system provides a module for entering caregiver information. Future development may include secure caregiver access, consent-based information sharing, and notifications.

5. Doctor Consultation

The application provides a basic interface for doctor consultation options. Future versions may support verified doctor profiles, appointment scheduling, and online consultation services.

6. Emergency SOS

The SOS module allows users to submit an emergency alert request through the application. Future development may include integration with verified emergency contacts and reliable notification services.

7. Tamil and English Language Support

The application aims to improve accessibility by supporting Tamil and English. This can help users who prefer interacting with healthcare applications in their local language.

8. Future Enhancements

The project can be extended with secure user authentication, mobile application support, medicine reminders, validated AI/ML-based health assistance, cloud deployment, and stronger data privacy mechanisms.

Overall Scope

Nalam AI aims to provide a centralized healthcare support platform for organizing health information and accessing basic guidance. The current version is a prototype; symptom guidance is predefined, and doctor consultation and SOS features require further integration before they can be relied upon in real-world situations. The application is intended to support healthcare access and organization, not replace professional medical diagnosis or emergency services.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

In many existing arrangements, healthcare services are provided through separate channels. A user may need one application to consult a doctor, another to remember medicines, and a different system to store medical records. Emergency communication may rely on manually calling family members or other contacts. These separate processes can make information difficult to organize and may be less convenient for users with limited digital experience.

The project material identifies the following limitations in existing systems:

- Healthcare services are often separated into individual solutions.
- A single integrated AI-enabled platform may not be available to the user.
- Language and accessibility barriers can affect rural and underserved communities.
- Medical history, medication information, and consultation details may not be organized in one place.
- Emergency notification and caregiver support may not be connected to other healthcare functions.

2.2 PROPOSED SYSTEM

NalamAI is proposed as an integrated AI-powered healthcare assistant. It combines a voice symptom checker, Tamil and English voice/text support, telemedicine or doctor consultation, medicine reminders, emergency SOS, caregiver notifications, and digital medical records. The integrated approach is intended to reduce the effort required to access multiple healthcare support functions.

The symptom checker accepts user-reported symptoms and provides preliminary health guidance. Users are encouraged to consult a qualified healthcare professional for diagnosis and treatment. Telemedicine provides a pathway for remote consultation. The medicine reminder records medication names, dosage, and timings so that users can receive reminders. The SOS module can alert registered emergency contacts and share location information when supported by the implementation. Digital records organize symptoms, medicines, and consultation information for later reference.

2.3 SYSTEM REQUIREMENTS SPECIFICATION

The exact deployment stack depends on the final implementation. The following are functional and platform-level requirements implied by the proposed system.

2.3.1 SOFTWARE REQUIREMENTS

- A web or mobile user interface for entering symptoms and viewing healthcare information.
- A backend service to process requests and manage module interactions.

- A data store for user profiles, medicine schedules, emergency contacts, and health records.
- Voice input/output support and Tamil-English text handling, where enabled.
- A notification mechanism for reminders and emergency alerts.
- A telemedicine connection or consultation workflow.

2.3.2 HARDWARE AND CONNECTIVITY REQUIREMENTS

- A smartphone, tablet, or computer with a supported browser or application.
- Internet connectivity for remote consultation and online data synchronization.
- Location permission and a supported location service for location sharing during SOS events.
- Secure access to the system and suitable protection for sensitive health information.

CHAPTER 3

LITERATURE SURVEY

3.1 REVIEW OF RELATED WORK

Digital health and telemedicine

The World Health Organization's Global Strategy on Digital Health 2020–2025 discusses the role of digital technologies in strengthening health systems. This provides a broad context for integrated systems such as NalamAI, where digital services are organized around user needs. WHO's report on telemedicine describes opportunities and developments in remote healthcare delivery, supporting the relevance of remote consultation for users who cannot easily reach healthcare facilities.

Artificial intelligence in healthcare

Topol's work on deep medicine examines the potential for artificial intelligence to support healthcare while keeping human care central. Davenport and Kalakota discuss opportunities for AI in healthcare, including support for clinical workflows and information handling. These sources provide context for using AI as an assistive component rather than treating automated output as a final medical diagnosis.

Deep learning and medical information

Esteva and colleagues review deep learning applications in healthcare and highlight the importance of suitable data and evaluation. Digital health systems must be designed carefully because errors can affect decisions about health. For NalamAI, this means symptom guidance should be presented as preliminary information and should direct users to qualified professionals when appropriate.

Smart healthcare monitoring

Research on smart healthcare monitoring in IoT environments explores how connected technologies may support health monitoring and information sharing. Although wearable monitoring is not a core feature of the current NalamAI proposal, it is identified as a possible future enhancement. Wearable devices could provide additional data if appropriate devices, consent, connectivity, and validation were available.

Health information systems and digital records

Health informatics literature describes the role of structured health information in supporting continuity of care. Digital records can help organize symptoms, medication information, and consultation history. NalamAI proposes a digital medical-record module so that important user information can be maintained for future reference, subject to appropriate privacy and access controls.

3.2 SUMMARY OF THE LITERATURE

The reviewed sources indicate that digital health, telemedicine, AI-assisted information services, and organized health records can contribute to more accessible healthcare when used

responsibly. The central design idea of NalamAI is to integrate these functions in one user-oriented platform. Important considerations include safety, data privacy, clear communication of system limitations, accessibility, and the availability of human healthcare professionals.

CHAPTER 4

SYSTEM ARCHITECTURE AND DESIGN

4.1 ARCHITECTURE OVERVIEW

The proposed architecture is organized into a user interaction layer, healthcare service modules, a data-management layer, and external services. Users provide information through voice or text. The system routes the request to the relevant module and presents a response or action. The modules share information only where required for the service, and sensitive records should be protected through authentication and access control.

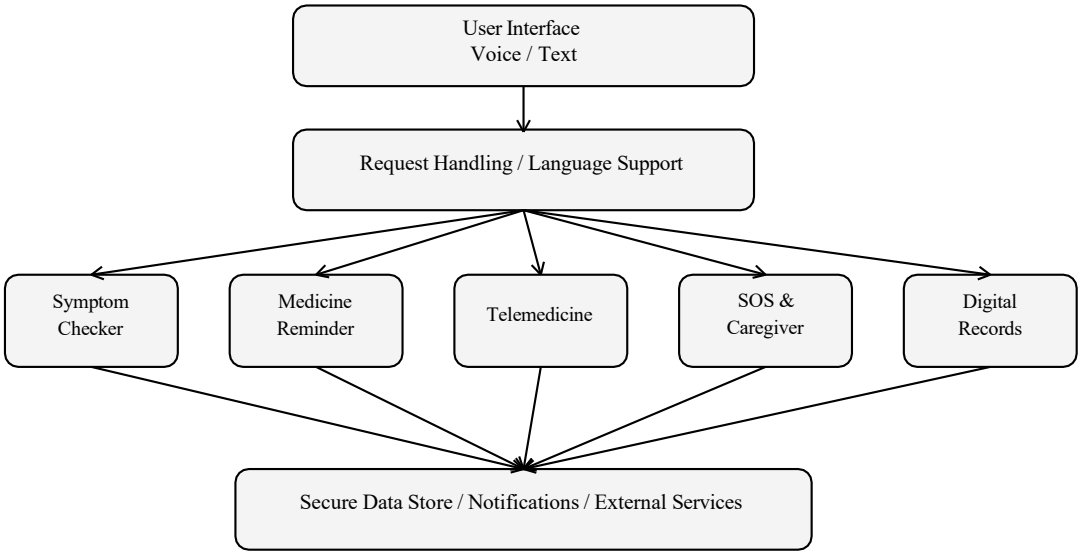


Fig. 4.1: Proposed NalamAI system architecture

4.2 MODULE DESCRIPTION

4.2.1 Symptom Checker Module

This module accepts symptoms described by the user through voice or text. It is intended to analyze the reported symptoms and provide preliminary health guidance. Tamil and English support can make the interaction more accessible. Because automated symptom analysis can be uncertain, the module should not claim to provide a confirmed diagnosis and should advise users to seek professional care for concerning symptoms.

4.2.2 Medicine Reminder Module

The medicine reminder module allows users to record medication names, dosage details, and scheduled timings. The system can use this information to generate reminders. The reminder function supports routine medication management but does not determine or change a prescribed dosage.

4.2.3 Telemedicine Module

The telemedicine module is intended to connect users with healthcare professionals remotely. It can provide a pathway for requesting or arranging a consultation. Actual consultation scheduling, video calling, and doctor availability depend on the services connected to the final implementation.

4.2 MODULE DESCRIPTION (CONTINUED)

4.2.4 Emergency SOS Module

The SOS module is designed to help a user notify predefined emergency contacts. When location services and permission are available, the alert may include the user's location. The module should clearly communicate whether an alert was successfully sent and should not be presented as a replacement for local emergency services.

4.2.5 Caregiver Notification Module

Caregiver notifications can share important health updates and emergency alerts with contacts authorized by the user. The information shared should be limited to what is needed and should respect the user's consent and privacy preferences.

4.2.6 Digital Medical Records Module

This module organizes symptoms, previous consultations, medicines, and other relevant health information. A structured record can help users refer to their own history and prepare for consultations. Access to records should be restricted to authorized users and protected against accidental disclosure.

4.3 WORKFLOW

The proposed workflow begins when the user opens NalamAI and selects a function. For symptom guidance, the user enters a symptom through voice or text; the system processes the input and displays preliminary guidance. For medication management, the user enters medicine details and timings, after which reminders can be generated. For telemedicine, the user follows the consultation pathway. For an emergency, the user activates SOS and the system attempts to notify registered contacts. Medical information can be saved in the digital record for future reference.

Fig. 4.2: NalamAI user workflow (conceptual)

User opens NalamAI → Selects module → Enters or reviews information → System processes request → Response, reminder, consultation pathway, or alert → Relevant information is stored securely when applicable.

4.4 DATA PRIVACY AND SAFETY

Health data is sensitive. A practical implementation should use secure authentication, access controls, protected transmission, and limited collection of personal information. Users should know what information is stored and who can access it. Voice or text symptom guidance must be framed as preliminary support. The application should not delay emergency care, and users with urgent symptoms should contact local emergency services or qualified medical professionals.

CHAPTER 5

IMPLEMENTATION APPROACH

5.1 FUNCTIONAL REQUIREMENTS

The following functional requirements describe the expected behaviour of the proposed system. They are design requirements; actual availability depends on the final software implementation.

ID	Requirement	Description
FR1	User input	The system should accept symptom descriptions through text and, where supported, voice.
FR2	Language support	The interface should support Tamil and English interaction.
FR3	Symptom guidance	The system should provide preliminary information based on user-reported symptoms and clearly state its limitations.
FR4	Medicine reminders	The system should store medicine name, dosage details, and timing entered by the user.
FR5	Telemedicine	The system should provide a path to remote doctor consultation.
FR6	SOS	The system should initiate alerts to registered contacts and share location when supported and permitted.
FR7	Caregiver support	The system should allow selected health updates or alerts to be shared with authorized caregivers.
FR8	Digital records	The system should maintain relevant symptoms, medication details, and consultation history.

5.2 NON-FUNCTIONAL REQUIREMENTS

S. No.	Attribute	Description
1	Usability	The interface should be simple and readable for users with different levels of digital experience.
2	Accessibility	Language and voice support should reduce barriers for users who prefer Tamil or English.
3	Reliability	Reminder and alert functions should communicate failures clearly.
4	Security	Health records and personal data should be protected from unauthorized access.
5	Maintainability	Modules should be designed so that improvements can be made without redesigning the entire platform.
6	Safety	Automated guidance should not be represented as a substitute for professional diagnosis.

5.3 IMPLEMENTATION PLAN

A modular implementation can be developed in stages. First, the user interface is created for the main modules. Next, data structures are defined for user details, medicines, emergency contacts, and

health records. The symptom checker can then be connected to a validated knowledge source or suitable model. Reminder scheduling and notifications are added, followed by the telemedicine workflow and SOS integration. Finally, the system is tested with sample data, usability checks, and privacy review.

1. Requirement Analysis

The first step is to identify the main requirements of the Nalam AI application. The system is designed to provide basic healthcare support through symptom checking, medicine reminders, medical record management, caregiver information, doctor consultation options, and emergency SOS functionality. The application supports Tamil and English interfaces.

2. Frontend Development

The frontend is developed using React.js, HTML, CSS, and JavaScript. It provides a user-friendly interface through which users can enter symptoms, view medicine details, manage medical records, and access other healthcare features. React state management is used to update the interface when users interact with the application.

3. Backend Development

The backend is developed using Node.js and Express.js. It handles requests from the frontend and provides API endpoints for users, medicines, medical records, caregivers, symptom checking, and SOS functionality. The backend validates incoming requests and returns responses to the frontend.

4. Database Integration

MongoDB is used to store application data. Mongoose is used to connect the backend with the database and manage data models. The database supports storing and retrieving user-related information, medicine details, medical records, and symptom-checking records.

5. Symptom Checker Implementation

The symptom checker accepts symptoms entered by the user and sends them to the backend through an API request. The current implementation returns predefined healthcare guidance. It is a prototype and does not use a trained machine-learning model to diagnose diseases.

6. Medicine and Medical Record Management

The medicine module allows users to enter medicine names, dosages, timings, and frequency. The medical records module allows users to save and retrieve record details. The frontend communicates with the backend APIs to perform these operations.

7. Caregiver and Emergency Modules

The caregiver module collects caregiver information and sends it to the backend for storage. The SOS module sends an emergency alert request to the backend. In the current prototype, these functions do not establish a verified connection to emergency services or guarantee that an alert reaches another person.

8. Testing and Validation

The application is tested by running the frontend and backend separately. API endpoints are checked to verify that requests and responses work correctly. Database operations are tested to confirm that

supported records can be saved and retrieved. The interface is also checked for navigation, input validation, and Tamil/English language switching.

9. Deployment and Maintenance

After successful testing, the frontend and backend can be deployed to suitable hosting platforms. The database connection and environment variables must be configured securely. Regular maintenance, error handling, data protection, and usability improvements are required for future development.

10. Expected Outcome

The expected outcome is a web-based healthcare support prototype that brings basic symptom guidance, medicine information, medical record management, caregiver details, and SOS request functionality into one application. The system is intended to improve accessibility and organization of healthcare-related information, not replace professional medical advice.

5.4 TESTING AND VALIDATION

Testing should confirm that each module behaves as expected under normal and error conditions. The cases below are suggested validation scenarios for the proposed system. They are not reported experimental results.

Test ID	Scenario	Expected Result
T01	Enter a symptom using text	System accepts input and returns preliminary guidance or a safe clarification.
T02	Submit an empty symptom field	System asks the user to enter symptom information.
T03	Enter medicine name and timing	Details are recorded and reminder settings can be reviewed.
T04	Use an unsupported language input	System explains supported languages or requests text input.
T05	Trigger SOS with contacts configured	System attempts to notify registered contacts and shows status.
T06	Open a saved health record	Authorized user can view stored information.
T07	Attempt access without authorization	System blocks access to private health information.
T08	Telemedicine service unavailable	System displays an error or alternative next step.

5.5 PERFORMANCE METRICS

Performance should be evaluated using both technical and user-centred measures. The exact values can only be reported after testing the implemented system.

S. No.	Metric	Description
1	Response time	Time taken to respond to a user request.
2	Reminder delivery rate	Percentage of scheduled reminders delivered as expected.
3	Alert delivery success	Percentage of SOS notifications successfully sent when connectivity and permissions are available.
4	Task completion rate	Percentage of users able to complete a selected task without assistance.
5	Input handling	Ability to handle incomplete, unclear, or unsupported input safely.
6	Record accuracy	Whether entered health information is stored and retrieved correctly.
7	User feedback	Feedback on clarity, usability, language support, and perceived usefulness.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

NalamAI is proposed as an integrated AI-powered smart healthcare assistant designed to improve access to healthcare support for rural and underserved communities. The system brings together symptom guidance, Tamil and English voice/text support, telemedicine, medicine reminders, emergency SOS, caregiver notifications, and digital medical records. The main objective is to reduce the need to use separate services for common healthcare support tasks and to provide a more convenient user experience.

The project highlights how AI and digital health tools can support information access and routine healthcare management. At the same time, the system must be implemented with careful attention to safety, privacy, accessibility, and reliability. Symptom guidance should remain preliminary, and qualified healthcare professionals must remain responsible for diagnosis and treatment. Emergency functions should clearly report whether an alert was delivered and should not replace local emergency services.

6.2 FUTURE ENHANCEMENTS

- Integrate wearable health-monitoring devices, after validating the accuracy and usefulness of the collected data.
- Add offline or low-connectivity functionality for selected features.
- Improve personalization while maintaining user consent and privacy.
- Expand language support based on community needs and careful testing.
- Connect to verified telemedicine providers and healthcare information sources.
- Introduce usability testing with representative users and healthcare professionals.
- Add stronger security controls, audit logs, and transparent data-management settings.

REFERENCES

[1] World Health Organization (WHO), *Global Strategy on Digital Health 2020–2025*. Geneva, Switzerland: WHO, 2021.

[2] World Health Organization, *Telemedicine: Opportunities and Developments in Member States*. Geneva, Switzerland: WHO Press, 2010.

[3] E. J. Topol, *Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again*. New York, NY, USA: Basic Books, 2019.

[4] T. Davenport and R. Kalakota, "The Potential for Artificial Intelligence in Healthcare," *Future Healthcare Journal*, vol. 6, no. 2, pp. 94–98, 2019.

[5] A. Esteva, A. Robicquet, B. Ramsundar, et al., "A Guide to Deep Learning in Healthcare," *Nature Medicine*, vol. 25, no. 1, pp. 24–29, 2019.

[6] E. B. Larson, "Building Trust in Artificial Intelligence for Healthcare," *The Lancet Digital Health*, vol. 1, no. 2, pp. e67–e68, 2019.

[7] M. M. Islam, M. Rahaman, and M. R. Islam, "Development of Smart Healthcare Monitoring System in IoT Environment," *SN Computer Science*, vol. 1, no. 3, pp. 1–11, 2020.

[8] R. K. Reddy and V. Kumar, "Artificial Intelligence in Healthcare: Opportunities and Challenges," *International Journal of Healthcare Information Systems and Informatics*, vol. 15, no. 3, pp. 1–15, 2020.

[9] J. G. Mantas and A. Hasman, *Health Informatics: An Overview*. Amsterdam, Netherlands: IOS Press, 2017.

[10] A. K. Jha, D. Doolan, D. Grandt, T. Scott, and D. W. Bates, "The Use of Health Information Technology in Seven Nations," *International Journal of Medical Informatics*, vol. 77, no. 12, pp. 848–854, 2008.

[11] S. Shickel, P. J. Tighe, A. Bihorac, and P. Rashidi, "Deep EHR: A Survey of Recent Advances in Deep Learning Techniques for Electronic Health Record Analysis," *IEEE Journal of Biomedical and Health Informatics*, vol. 22, no. 5, pp. 1589–1604, 2018.

[12] D. W. Bates, S. Saria, L. Ohno-Machado, A. Shah, and G. Escobar, "Big Data in Health Care: Using Analytics to Identify and Manage High-Risk and High-Cost Patients," *Health Affairs*, vol. 33, no. 7, pp. 1123–1131, 2014.

[13] E. Ammenwerth and C. Iller, "IT-Adoption and the Interaction of Task, Technology and Individuals," *BMC Medical Informatics and Decision Making*, vol. 6, no. 3, pp. 1–9, 2006.

[14] D. Luxton, *Artificial Intelligence in Behavioral and Mental Health Care*. London, U.K.: Academic Press, 2016.

[15] H. Yang, J. Yu, and X. Luo, “Artificial Intelligence Enabled Health Monitoring Systems,” *IEEE Access*, vol. 8, pp. 123456–123470, 2020.

[16] A. Bohr and K. Memarzadeh, *Artificial Intelligence in Healthcare*. London, U.K.: Academic Press, 2020.

[17] J. Hollis, C. Falconer, M. Martin, et al., “Annual Research Review: Digital Health Interventions for Children and Young People,” *Journal of Child Psychology and Psychiatry*, vol. 58, no. 4, pp. 474–503, 2017.

[18] S. Patel, H. Park, P. Bonato, L. Chan, and M. Rodgers, “A Review of Wearable Sensors and Systems with Application in Rehabilitation,” *Journal of NeuroEngineering and Rehabilitation*, vol. 9, no. 21, pp. 1–17, 2012.

[19] P. Groves, B. Kayyali, D. Knott, and S. Van Kuiken, *The Big Data Revolution in Healthcare*. McKinsey & Company, 2013.

[20] A. Sheikh, K. Sood, and S. Bates, “Leveraging Health Information Technology to Achieve the Triple Aim of Healthcare Reform,” *Journal of the American Medical Informatics Association*, vol. 22, no. 4, pp. 849–856, 2015.

[21] J. Jiang, B. Guo, Y. Li, and J. Chen, “The Future of Healthcare with Artificial Intelligence,” *Healthcare Technology Letters*, vol. 4, no. 5, pp. 1–10, 2018.

[22] S. Ryu, “Telemedicine: Opportunities and Developments in Member States,” *Healthcare Informatics Research*, vol. 18, no. 2, pp. 153–155, 2012.

[23] R. Agarwal, G. Gao, C. DesRoches, and A. K. Jha, “Research Commentary—The Digital Transformation of Healthcare,” *Information Systems Research*, vol. 21, no. 4, pp. 796–809, 2010.

[24] P. T. Cortez and M. Morais, “Electronic Health Records and Their Impact on Healthcare Quality,” *International Journal of Medical Informatics*, vol. 82, no. 5, pp. 321–330, 2013.

[25] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

APPENDIX I – ILLUSTRATIVE LOGIC

The following Python example illustrates a simple rule-based demonstration flow for symptom input. It is not a medical diagnostic model and is not a complete implementation of NalamAI's telemedicine, reminder, SOS, or medical-record modules. Any real symptom checker should be reviewed and validated by qualified professionals.

```
import { useState, useEffect } from "react";
import "./App.css";

function App() {
 const [language, setLanguage] = useState("en");
 const [symptoms, setSymptoms] = useState("");
 const [analysis, setAnalysis] = useState("");
 const [medicines, setMedicines] = useState([]);
 const [records, setRecords] = useState([]);

 const userId = "USER_ID";

 useEffect(() => {
  fetch(`http://localhost:5000/api/medicines/${userId}`)
   .then(res => res.json())
   .then(data => setMedicines(data))
   .catch(console.error);

  fetch(`http://localhost:5000/api/medical-records/${userId}`)
   .then(res => res.json())
   .then(data => setRecords(data))
   .catch(console.error);
 }, []);

 async function analyzeSymptoms() {
  const res = await fetch(
   "http://localhost:5000/api/symptoms/analyze",
   {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ symptoms })
   }
  );

  const data = await res.json();
  setAnalysis(data.analysis);
 }

 async function saveMedicine(medicine) {
  const res = await fetch(
  "http://localhost:5000/api/medicines",
   {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ user: userId, ...medicine })
   }
```

```
  );

  const data = await res.json();
  if (res.ok) setMedicines([...medicines, data]);
}

async function saveRecord(record) {
  const res = await fetch(
    "http://localhost:5000/api/medical-records",
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ user: userId, ...record })
    }
  );

  const data = await res.json();
  if (res.ok) setRecords([...records, data]);
}

return (
  <div>
    <h1>Nalam AI</h1>

    <button onClick={() =>
      setLanguage(language === "en" ? "ta" : "en")
    }>
      {language === "en" ? "தமிழ்" : "English"}
    </button>

    <h2>Symptom Checker</h2>
    <input
      value={symptoms}
      onChange={e => setSymptoms(e.target.value)}
      placeholder="Enter your symptoms"
    />
    <button onClick={analyzeSymptoms}>Analyze</button>
    <p>{analysis}</p>

    <h2>Medicine Reminder</h2>
    <ul>
      {medicines.map((m, i) => (
        <li key={m._id || i}>
          {m.medicineName} - {m.dosage}
        </li>
      ))}
    </ul>

    <h2>Medical Records</h2>
    <ul>
      {records.map((r, i) => (
        <li key={r._id || i}>
          {r.title}: {r.details}
        </li>
```

```
        ))}
      </ul>
    </div>
  );
}

export default App;
const express = require("express");
const SymptomCheck = require("../models/SymptomCheck");

const router = express.Router();

// Save symptom check
router.post("/", async (req, res) => {
try {
    const savedCheck = await new SymptomCheck(req.body).save();
    res.status(201).json(savedCheck);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});

// Analyze symptoms
router.post("/analyze", async (req, res) => {
try {
    const { symptoms } = req.body;

    if (!symptoms) {
      return res.status(400).json({
        message: "Symptoms are required"
      });
    }

    res.json({
      symptoms,
      analysis:
        "Please rest, stay hydrated, monitor your symptoms, " +
        "and consult a doctor if symptoms persist or become severe.",
      urgency: "medium"
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Get symptom history
router.get("/:userId", async (req, res) => {
try {
    const checks = await SymptomCheck.find({
      user: req.params.userId
    }).sort({ createdAt: -1 });

    res.json(checks);
  } catch (error) {
```

```
    res.status(400).json({ message: error.message });
  }
});

module.exports = router;
```

Suggested demonstration steps:

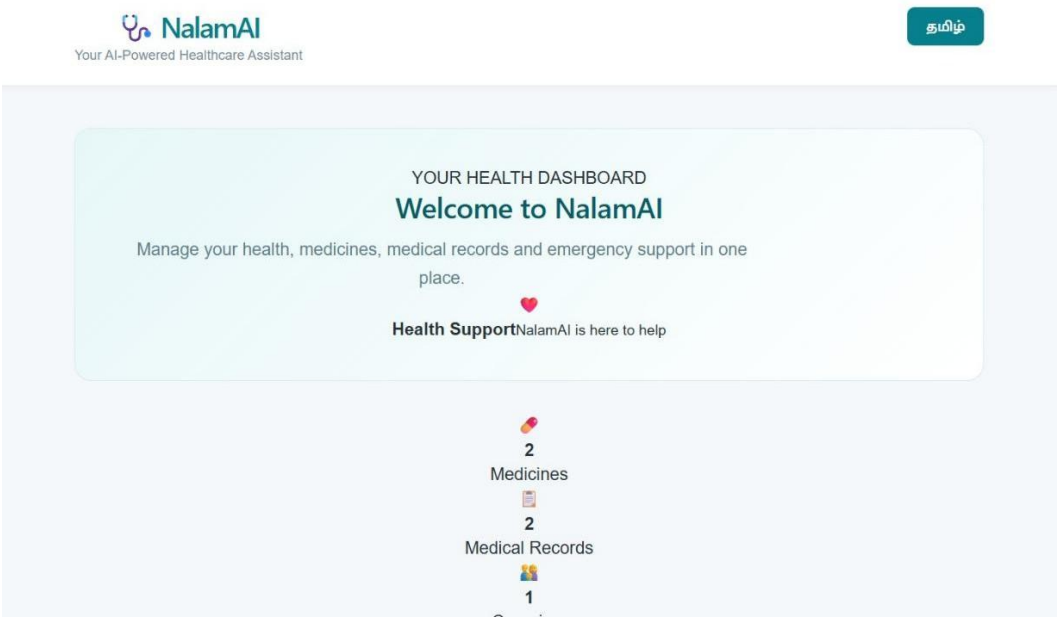
- Run the script in a Python terminal.
- Enter a general symptom such as “headache” and observe the safety-oriented response.
- Enter a possible urgent symptom such as “chest pain” and observe the emergency guidance.
- Enter no text to verify input validation.

This example uses simple keyword checks only. It does not predict diseases, store health data, send real alerts, or connect to doctors.

APPENDIX II – MODULE SCREENSHOT PLACEHOLDERS

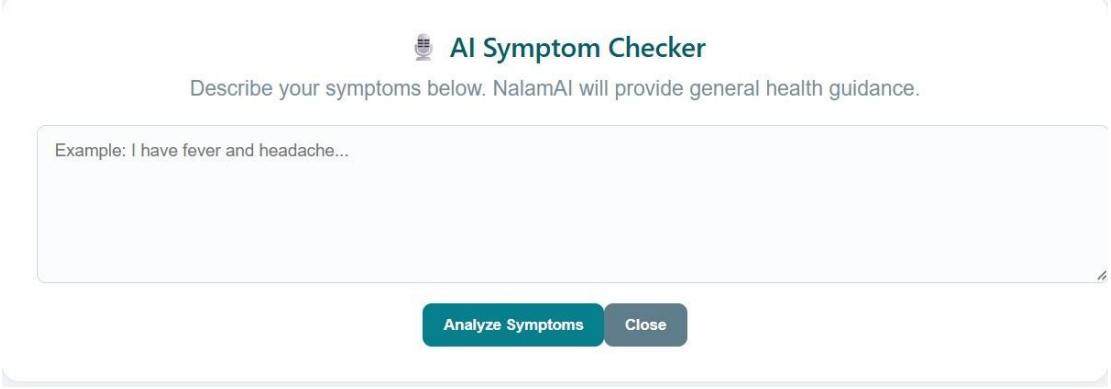
The source NalamAI document provided for this report describes the proposed modules but does not include verified application screenshots or implementation evidence. Add screenshots from the actual working application in the spaces below after implementation and testing.

1. Home page / module selection

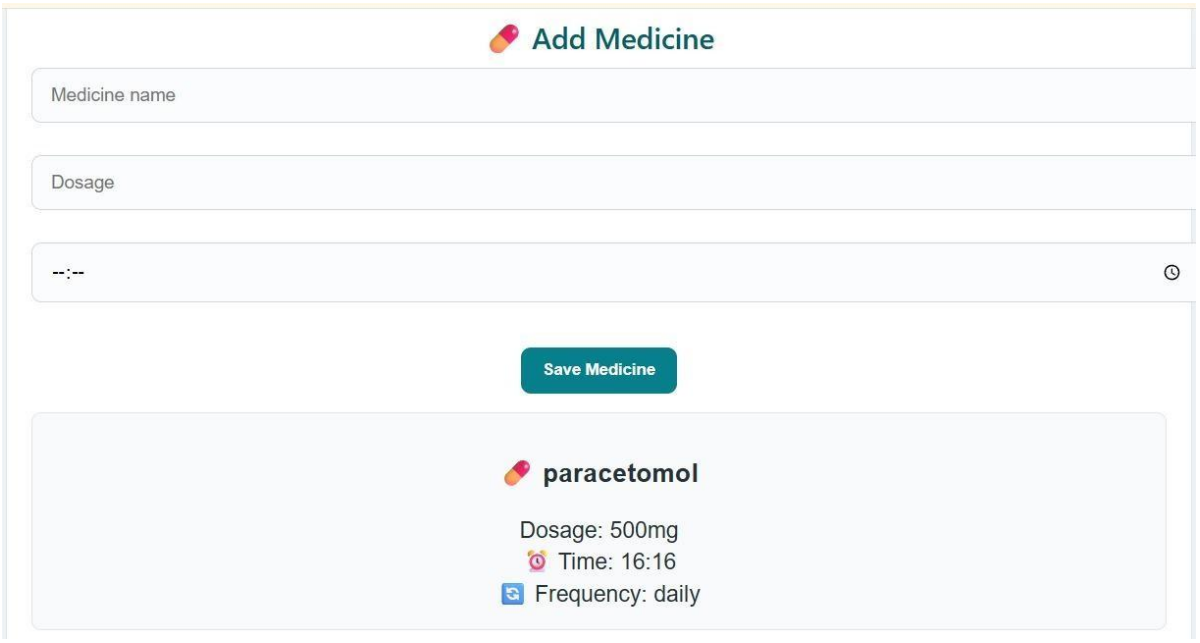


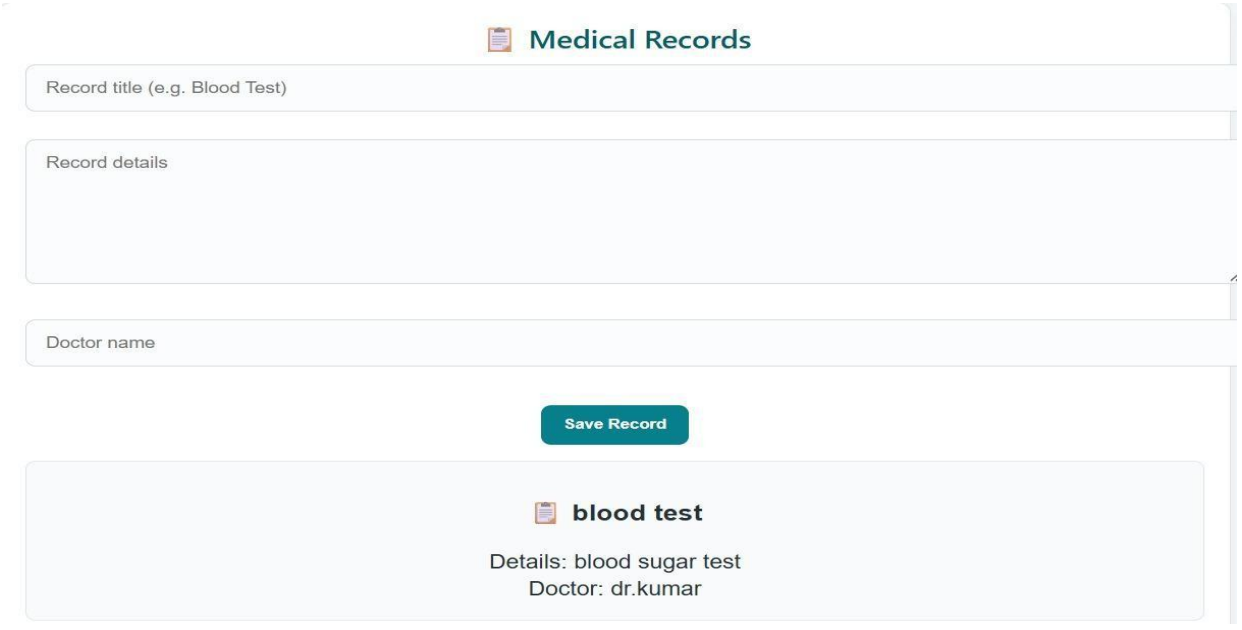
---

2. Symptom checker screen



3. Medicine reminder / SOS / medical record screen







THANK YOU

NALAMAI – AI-POWERED SMART HEALTHCARE ASSISTANT